

PathFinder Backend Resume Processing & Fallback Workflow

This document explains the technical pipeline that runs in the backend when a user uploads a resume, along with how PathFinder handles errors and fallbacks at every stage.

1. Pipeline Overview

The diagram below shows the order in which data is processed:

[Architecture Flowchart - See Interactive SVG/Web Version]

2. Step-by-Step Backend Pipeline & Fallback Mechanisms

Stage 1: Text Extraction (Reading the File)

- **What happens:** The file stream is passed to the parser helper in resume_parser.py.
- **Primary Path:**
 - **PDFs:** Parsed using the ``pypdf`` library, extracting text page-by-page.
 - **DOCX:** Unzipped in memory to parse the inner XML tag tree (``word/document.xml``) for paragraphs.
- **If it fails:**
 - * If ``pypdf`` crashes (e.g. because of non-standard file encoding), the parser triggers a **custom fallback byte-stream parser**. This scanner searches for raw PDF ``stream`` tags, decompresses them using ``zlib``, and extracts matching text blocks using regular expressions.
 - **Ultimate Fallback:** If both options fail, the backend catches the exception, halts the scan, and alerts the user: **"Failed to parse file. Please copy-paste your skills manually."**

Stage 2: Skill Extraction (Identifying Superpowers)

- **What happens:** The raw text document is sent to skill_extractor.py to find technical and soft skills.
- **Primary Path:**
 - * If a **Groq API Key** is active, the text is sent to the ``llama-3.1-8b-instant`` model. The LLM acts as an editor, identifying clean skill terms (e.g. converting "writing scripts in Python" to just ``Python``).
- **If it fails (Fallback Path):**
 - * If the API key is missing, invalid, or the Groq network request times out, the backend falls back to the **Local rule-based extractor**.
 - * This matches words against the vocabulary of 515 skills in skills.csv using regular expressions and ``spaCy`` word tokenizers.

Stage 3: Career Matcher (Finding the Job)

- **What happens:** The cleaned list of skills is sent to career_recommender.py.

- **Primary Path:**
 - * If a **Groq API Key** is available, the LLM evaluates the profile context (skills + interests) to recommend a career and calculate a compatibility score.
- **If it fails (Fallback Path):**
 - * If the API is offline or keyless, the system falls back to **Local Cosine Similarity Math**.
 - * The system vectorizes the skills using `TfidfVectorizer`` and compares the user's vector to 15 standard career profiles loaded from `careers.csv`.
- **Secondary CSV Fallback:** If the ``careers.csv`` file itself is missing, the code catches the error and loads a default hardcoded career list (`DEFAULT_CAREERS``) defined in python memory.

Stage 4: Educational Resource Advisor (Finding Courses & Certificates)

- **What happens:** The system queries datasets to recommend courses and certifications.
- **Primary Path:**
 - * The career name is embedded and queried against a local **ChromaDB Vector Database** containing indexed resources. This allows semantic matching (e.g. matching "Machine Learning" query to a course named "Neural Networks").
- **If it fails (Fallback Path):**
 - * If ChromaDB is not installed, or the database folder is corrupted, the system falls back to **TF-IDF Vector Fallback Search**.
 - * It loads `couses.csv` and `certifications.csv` into Pandas dataframes and finds matches by searching the text fields mathematically.

Stage 5: Roadmap Generator (Creating the Timeline)

- **What happens:** Creates the week-by-week learning goals.
- **Primary Path:**
 - * If Groq is online, the LLM dynamically designs a customized learning plan tailored to the user's current skills, gaps, and interests.
- **If it fails (Fallback Path):**
 - * If Groq fails, the system loads a pre-written learning roadmap from a local dictionary database ``LOCAL_ROADMAPS`` matching the recommended career.
- **Ultimate Fallback:** If the recommended career is not one of the pre-written profiles in the dictionary, the system assigns a generic 20-week learning progression (`DEFAULT_ROADMAP``) covering baseline industry preparation.

Summary of Fallback Priorities

System Step	Primary Method	Fallback Method	Ultimate Fail-safe
File Parsing	<code>`pypdf` / Docx parser</code>	Custom binary stream parser	UI alert + manual paste prompt
Skill Extraction	Groq LLM API (Llama-3)	Local Regex / <code>`spaCy`</code> match	Uses raw manual text
Career Matching	Groq LLM API (Llama-3)	TF-IDF Cosine Similarity	Uses hardcoded default career profiles
Resource Search	ChromaDB semantic search	TF-IDF Fallback search	Displays empty lists without crashing
Roadmap Plan	Groq LLM API (Llama-3)	Predefined <code>`LOCAL_ROADMAPS`</code>	Generic 4-stage upskilling template